

A Landmark-Based Transformation Approach to Concurrency Control in Group Editors

Rui Li and Du Li
Department of Computer Science
Texas A&M University
College Station, Texas 77843-3112 USA
Email Contact: lidu@cs.tamu.edu

ABSTRACT

Operational transformation (OT) is a responsive and non-blocking concurrency control method widely-accepted in group editors. Correctness and performance are the basis of usefulness and usability of OT-based group editors. However, the correctness of previous OT algorithms depends on conditions that are very difficult to verify. In this paper we propose a novel landmark-based transformation (LBT) approach, its correctness no longer depending on those conditions and thus easy to prove. In addition, we give an example algorithm that significantly outperforms a state-of-the-art OT algorithm. This work reveals a more practical approach to developing OT algorithms.

Categories and Subject Descriptors

H.5.3 [Information Systems]: Group and Organization Interfaces—*Synchronous Interaction, Theory and Models*; G.2 [Mathematics of Computing]: Discrete Mathematics

General Terms

Algorithms, Theory, Verification

Keywords

Concurrency Control, Data Consistency, Groupware, Group Editor, Operational Transformation

1. INTRODUCTION

Group editing is a classic groupware topic that has been actively researched in the computer-supported cooperative work field over the past two decades. A group editor allows a group of geographically separated human users to edit a shared document over a computer network. To hide the communication latencies and achieve fast local responsiveness, the shared document is often replicated at cooperating sites. A group editor allows the users to work on the local

replicas of the document, consistency among which is maintained by a concurrency control algorithm.

A special concurrency control method called operational transformation (OT) [2, 13] has been well established in this context. OT allows for responsive and nonblocking interaction as local operations are always executed immediately once generated. Remote operations are *transformed* before execution to repair inconsistencies. Every OT algorithm has two parts: a set of (inclusion and exclusion) transformation functions, which address how to transform two operations, and a control procedure, which addresses how to call these functions to integrate remote operations [13].

A large number of OT algorithms have been proposed over the past 15 years, e.g., [1, 2, 9, 10, 11, 14, 15]. The correctness of these algorithms is generally subject to the criteria given in [11, 14], which include a condition (intention preservation) that is difficult to verify in practice. In addition, mainly under the influence of [9], the inclusion transformation functions in these algorithms are generally required to satisfy two properties that are also extremely difficult to verify. As a temporary resort, several approaches are proposed to avoid verification of these properties, e.g., [8, 10, 14, 15]. Nonetheless, their correctness still must verify intention preservation. These problems call for a more practical approach to developing and proving OT algorithms.

In this paper, we propose a novel approach called landmark-based transformation (LBT). The approach first formally establishes a set of sufficient conditions of transformation functions and then finds a control algorithm that can ensure these sufficient conditions. These conditions are based on a concept of landmark that is derived from the correctness criteria given in [6]. Consequently OT algorithms developed in this approach are easy to prove and no longer need to verify properties in [9] and the intention preservation condition in [11, 14]. As a by-product, the example control algorithm given in this paper significantly outperforms the state-of-the-art algorithm in [6]. This improvement is important for cooperative work because it directly affects the responsiveness and ultimately the usability of group editors that implement OT for concurrency control.

The rest of this paper is organized as follows. First we give a background of OT in Section 2. Then Section 3 explains the correctness criteria, Section 4 establishes sufficient conditions of transformation functions, and Section 5 gives a control algorithm. Section 6 shows an example of the algorithm. After that, Section 7 compares related work and Section 8 summarizes contributions of this paper.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

GROUP'05, November 6-9, 2005, Sanibel Island, Florida, USA.

Copyright 2005 ACM 1-59593-223-2/05/0011...\$5.00.

2. BACKGROUND

As a convention in group editors [2], we model the shared document as a string s . To distinguish different states at different sites, we annotate it like s_i^n , where i is the site id and n is the state number. The superscript or subscript is often omitted if it is not interesting. Suppose the first character of any nonempty string is indexed as zero. Also as a convention we use the following two primitive operations: $\text{ins}(p, c)$, which inserts a character c at position p , and $\text{del}(p)$, which deletes the character at position p .

Apparently the position parameter p of any operation o is defined relative to some state s . Hence we often annotate o with its *definition state* s like o^s . If o^s is generated in s , then s is called its *generation state*. If o^s is to be executed in s , then s is called its *execution state*. An operation's execution state is not necessarily equivalent to its generation state if the operation is executed at a remote site [13].

Given operation o , function $p(o)$ returns its position value, $t(o)$ returns its operation type (*ins* or *del*), $c(o)$ returns the effect character to be inserted or deleted, $\text{id}(o)$ returns the id of the site that generates o , $\text{gst}(o)$ denotes its generation state, and $\text{est}(o)$ denotes its execution state. The fact that the execution of o in state s yields s' is denoted by $\text{exec}(s, o) = s'$. We use the following two relations, $\rightarrow$ and $\parallel$, without further definition as they have long been established [2]. For any two operations o_1 and o_2 , $o_1 \rightarrow o_2$ iff o_1 happened before o_2 , and $o_1 \parallel o_2$ iff o_1 is concurrent with o_2 .

The basic idea of OT is to execute any local operation as soon as it is generated but transform a remote operation against concurrent operations that have been executed locally before its execution. Hence a history buffer HB is maintained at each site to keep track of all executed (local and remote) operations in their order of execution.

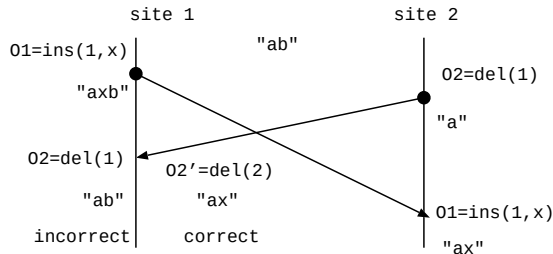


Figure 1: OT transforms o_2 and then executes o'_2 .

As a simple example, consider the scenario in Figure 1. Suppose two sites start from the same initial state $s_1^0 = s_2^0 = \text{"ab"}$. Site 1 performs $o_1 = \text{ins}(1, x)$ to insert character 'x' before 'b', yielding $s_1^1 = \text{"axb"}$, while site 2 concurrently performs $o_2 = \text{del}(1)$ to delete character 'b', yielding $s_2^1 = \text{"a"}$. When $o_2^0 = \text{del}(1)$ arrives at site 1, if it is executed as it is, then the wrong character 'x' will be deleted. This is because o_2^0 is generated in s_2^0 without the knowledge of o_1 , but its execution state s_1^1 has been changed by the execution of o_1 , which invalidates its position parameter. Hence we must shift the position of o_2^0 to incorporate the effect of o_1 such that the result o_2^1 can be correctly executed in state s_1^1 . This process is called *inclusion transformation* (IT) [14] (or forward transposition [11]).

Because a character has been inserted by o_1^1 on the left

of its intended position, o_2 should delete the character currently at position 2 instead of 1, i.e., $o_2^1 = \text{IT}(o_2^0, o_1^1) = \text{del}(2)$. The execution of o_2^1 in s_1^1 leads to the correct state $s_1^2 = \text{"ax"}$, which is identical to the final state at site 2 after o_2 and o_1 are executed in a tandem. As a result, OT seems able to achieve convergence and operation intentions despite the different orders of execution at different sites.

Another type of transformation function in the literature is called *exclusion transformation* (ET) [14] (or backward transposition [11]). Suppose $\text{exec}(s^1, o_1^1) = s^2$. Function $\text{ET}(o_2^2, o_1^1)$ returns o_2^1 , which *excludes* the effect of o_1^1 from o_2^2 as if o_1^1 had not been executed earlier in s^1 , such that the result o_2^1 is defined and can be executed in s^1 . In the above example, we have $o_2^0 = \text{ET}(o_2^1, o_1^1) = \text{del}(1)$.

It has been generally accepted [13] that each OT algorithm consists of two parts: a set of *transformation functions* (such as IT and ET) that determine how one operation is transformed against another, and a *control algorithm* that determines how an operation is transformed against a given operation sequence (e.g., the history buffer).

3. CONSISTENCY MODEL

The consistency model on which this work is based has been presented in [4, 6]. It is not to be claimed as a contribution in this paper. For this paper to be self-contained, however, we will explain the main concepts and argue that our notion of operation effects relation $\prec$ is a necessary constraint on group editors. A new concept of landmark character set will be introduced, which will be used later.

Observe that characters in any string can be uniquely identified by their positions. As a convention in group editors [2, 13], we assume that every character in a string is an unique object, possibly with different attributes such as color and boldness. The ASCII code of a character may only be one of the object's attributes. Due to this assumption, any two appearances of the same character have different ids and no two insertions insert the same character. Given any non-empty string s and character c is the i^{th} element in s , we denote $s[c] = i$ and $c = s[i]$. For convenience we often use s to denote the set of characters in s .

We first define the concept of **character relation** $\prec$ as follows: Assume that two special invisible characters c_b, c_e mark the beginning and end of any string s , respectively. For any string s and two character $c_1, c_2 \in s$, we say $c_1 \prec c_2$ iff $s[c_1] < s[c_2]$. Apparently $\prec$ is transitive and a total order. When a new character c is inserted into position p of s , the insertion establishes relation between c and the two existing characters $s[p-1]$ and $s[p]$ such that $s[p-1] \prec c \prec s[p]$. When a character is deleted from position p of s , it does not change the established relation $\prec$.

Assuming that all sites in a group editing session start from the same initial state s^0 and the same relation $\prec$ defined on s^0 . Relation $\prec$ evolves as operations are executed in their generation and execution states. When an operation o is executed locally in $\text{gst}(o)$, we can safely assume that its execution is correct and the relation $\prec$ resulted from its execution reflects the intention of o . When o is executed at a remote site, the characters in $\text{est}(o)$ may not be the same as those in $\text{gst}(o)$, due to concurrent execution of other operations. However, the execution of o must not violate the relation that has been established as it was executed in $\text{gst}(o)$.

That is, if o is an insertion, its execution must preserve the relation $\prec$ on all characters that appear in both $\text{est}(o)$ and $\text{gst}(o)$. If o is a deletion, it must delete the same character in $\text{est}(o)$ as in $\text{gst}(o)$.

In addition, the execution of o must not violate the partial relations that have been respectively established when o and its concurrent operations are executed in their generation states. Therefore we need a sort of “global” view that can combine these pieces into a whole relation. To achieve so, we must introduce two tie-breaking policies: First, when two insertions o_1, o_2 concurrently insert between the same two characters c_x, c_y in their generation states, i.e., $c_x \prec c(o_1) \prec c_y$ and $c_x \prec c(o_2) \prec c_y$, we enforce that the one with a smaller site id precedes the other. That is, if $\text{id}(o_1) < \text{id}(o_2)$, the resulted relation will be $c_x \prec c(o_1) \prec c(o_2) \prec c_y$. Secondly, when an operation o_1 inserts a character in $\text{gst}(o_1)$ at the same position as a character that was deleted by o_2 , i.e., $\exists c_x, c_y \in \text{gst}(o_1) : c_x \prec c(o_1) \prec c_y$ and $c_x \prec c(o_2) \prec c_y$, we enforce that the inserted character precedes the deleted character in relation $\prec$. That is, $c_x \prec c(o_1) \prec c(o_2) \prec c_y$.

Due to these two policies, when any operation o is to be executed in $\text{est}(o)$, all relations that have been established by o in $\text{gst}(o)$ and operations executed so far at this site can be merged as a total order. More generally, if we use C_t to denote the set of characters that ever appear in a group editing session, relation $\prec$ defined over this set is a total order. We state this total order in the following lemma with its proof omitted.

LEMMA 1. *Relation $\prec$ is a total order over C_t , i.e., $\forall c_1, c_2 \in C_t$: either $c_1 \prec c_2$ or $c_2 \prec c_1$.*

The **operation effects relation** can be considered a subset of the above explained character relation, with the part involving characters in the initial state removed. Hence we use the same notation $\prec$ for operation effects relation. Note there are two cases in which the effect characters of two given operations o_1, o_2 are equivalent or $c(o_1) \equiv c(o_2)$: o_1 and o_2 concurrently delete the same character, and o_1 deletes the same character inserted by o_2 .

DEFINITION. 1. *A group editor is correct if it always maintains the following two properties:*

1. Causality preservation: *For any two operations o_1 and o_2 , if $o_1 \rightarrow o_2$, then o_1 is executed before o_2 at all sites.*
2. Operation effects relation preservation: *The execution of any operation in any state preserves the operation effects relation $\prec$.*

It is shown in [4, 6] that preservation of operation effects relation implies convergence. Hence the above-defined correctness criteria no longer include convergence as do previous criteria [2, 9, 11, 14]. In previous work, causality preservation, convergence, and intention preservation are proposed as correctness criteria of group editors, including [11, 13, 14, 15]. However, due to the under-formalization of the intention preservation condition, no previous OT algorithms have been proved to preserve intentions to our knowledge. Counterexamples given in [4, 6] have shown that no previous OT algorithms addressed the operation effects relation violation (ERV) problem, including SOCT2 [11], GOT [14], GOTO [13], SOCT3/4 [15], and IMOR [3], and that this problem does not depend on the tie-breaking policies.

It is worth noting that relation $\prec$ is not an artificial constraint that group editors do not have to satisfy. The relation is mostly determined as operations are generated and executed in their generation states. The only “artificial” part is the two policies that are used to break ties, as explained above. Policies as such are widely used in distributed computing systems and OT algorithms, including dOPT [2], SOCT2/3/4 [11, 15], and GOT/GOTO [13, 14]. Compared to the intention preservation condition in previous work, our notion of operation effects relation preservation is easier to formalize and verify. Hence it is a more plausible correctness condition for group editors and OT algorithms.

To determine the relation between any two given characters, if we know whether or not there exists at least one character between them, the problem becomes easier.

DEFINITION. 2. [**Landmark Character**] *For any three characters $c_1, c_2, c_3 \in C_t$, we say that c_3 is a landmark between c_1 and c_2 , iff either $c_1 \prec c_3 \prec c_2$ or $c_2 \prec c_3 \prec c_1$.*

We denote the set of landmark characters between c_1 and c_2 as $C_{ld}(c_1, c_2) = \{c_3 \in C_t | c_1 \prec c_3 \prec c_2 \vee c_2 \prec c_3 \prec c_1\}$. It is abbreviated as C_{ld} when there is no confusion in the context. Since it is the existence of C_{ld} that matters, we do not really need to compute C_{ld} .

4. TRANSFORMATION FUNCTIONS

In this section, we define the IT/ET functions based on relation $\prec$ and discuss their sufficient conditions under which relation $\prec$ can be correctly determined by only using the basic operation parameters $p(o)$, $\text{id}(o)$, and $t(o)$. We assume there is a hash table ER at each site which is used to save the locally derived effects relation.

4.1 IT and Preconditions

The purpose of $o_1^{s'} = \text{IT}(o_1^s, o_2^s)$, where $s = \text{est}(o_1) = \text{est}(o_2)$ and $s' = \text{exec}(s, o_2^s)$, is to incorporate the effect of o_2^s into o_1^s such that the result $o_1^{s'}$ can be correctly executed in state s' . As defined in Algorithm 1, we first call Algorithm 2 (`get_it_er`) to determine the effect relation of o_1 and o_2 , and then use this relation to decide how to compute $o_1^{s'}$.

Algorithm 1 $\text{IT}(o_1^s, o_2^s) : o_1^{s'}$

```

1: get_it_er( $o_1, o_2$ )
2: if  $c(o_1) \equiv c(o_2)$  then
3:   return  $o_1' \leftarrow \phi$ 
4: else
5:    $o_1' \leftarrow o_1$ 
6:   if  $c(o_2) \prec c(o_1)$  then
7:     if  $t(o_2) = \text{ins}$  then
8:        $p(o_1') \leftarrow p(o_1) + 1$ 
9:     else  $\{ t(o_2) = \text{del} \}$ 
10:       $p(o_1') \leftarrow p(o_1) - 1$ 
11:   end if
12: end if
13: return  $o_1'$ 
14: end if
```

Assuming that every insertion introduces a new character, it is impossible that o_1 has the same effect character as another operation o_2 . However, two concurrent deletions

may intend to delete the same existing character. To handle this situation, we adopt a policy in which this character is only deleted once. The deletion received later is transformed into a null operation ϕ , which will not be executed. This corresponds to lines 1-2 in Algorithm 1 and lines 8-9 in Algorithm 2.

If $c(o_1) \prec c(o_2)$, it means that $c(o_2)$ is on the right side of $c(o_1)$ and the execution of o_2 does not affect o_1 . Hence o_1 is returned as it is. In the case of $c(o_2) \prec c(o_1)$, however, if o_2 inserted a character on the left of $c(o_1)$, the position of o_1 should be increased by one; or if o_2 deleted a character on the left, the position of o_1 should be decreased by one.

Algorithm 2 $\text{get_it_er}(o_1, o_2)$: relation of $c(o_1)$ and $c(o_2)$

```

1: return the relation if it is found in  $ER$ 
2:  $er \leftarrow c(o_2) \prec c(o_1)$ 
3: if  $p(o_1) < p(o_2)$  then
4:    $er \leftarrow c(o_1) \prec c(o_2)$ 
5: else if  $p(o_1) = p(o_2)$  then
6:   if  $t(o_1) = t(o_2) = \text{ins} \wedge \text{id}(o_1) < \text{id}(o_2)$  then
7:      $er \leftarrow c(o_1) \prec c(o_2)$ 
8:   else if  $t(o_1) = t(o_2) = \text{del}$  then
9:      $er \leftarrow c(o_1) \equiv c(o_2)$ 
10:  else if  $t(o_1) = \text{ins} \wedge t(o_2) = \text{del}$  then
11:     $er \leftarrow c(o_1) \prec c(o_2)$ 
12:  end if
13: end if
14: record  $er$  in  $ER$ 
15: return  $er$ 

```

It is required that the two operations in IT be defined relative to the same state, i.e., $\text{est}(o_1) = \text{est}(o_2)$, for, otherwise, it does not make sense to compare their position parameters [13, 14]. However, condition $\text{est}(o_1) = \text{est}(o_2)$ alone is not sufficient for correctly determining the relation between $c(o_1)$ and $c(o_2)$ in IT. There are well-known scenarios in which this condition is satisfied but the relation cannot be correctly determined by only using position parameters [6, 11, 14]. We define a set of rules in Algorithm 2 for determining the effect relation between any two given operations. In the following lemmas, we study the sufficient conditions of get_it_er by cases and then draw conclusions in Corollary 1. The preconditions of IT are exactly the same.

LEMMA 2. *$\text{get_it_er}(o_1, o_2)$ is correct if (1) $t(o_1) = t(o_2) = \text{ins}$, (2) $s = \text{est}(o_1) = \text{est}(o_2)$, (3) $C_{ld}(c(o_1), c(o_2)) \subseteq s$.*

Proof: Condition $C_{ld}(c(o_1), c(o_2)) \subseteq s$ means that the landmark characters between $c(o_1)$ and $c(o_2)$, if any, must be present in s . If $|C_{ld}| \neq 0$, then there exists at least one character $c \in C_{ld}$ between $p(o_1)$ and $p(o_2)$ in s . By definition 2, we have either $c(o_1) \prec c \prec c(o_2)$ or $c(o_2) \prec c \prec c(o_1)$. Without loss of generality, suppose $c(o_1) \prec c \prec c(o_2)$. By transitivity we have $c(o_1) \prec c(o_2)$. Since $c(o_2)$ and c are already in s and $c(o_1)$ is to be inserted before c , we have $p(o_1) \leq s[c] < p(o_2)$, which implies $p(o_1) < p(o_2)$. In get_it_er , if $p(o_1) < p(o_2)$, we get $c(o_1) \prec c(o_2)$.

If $|C_{ld}| = 0$, by Definition 2, there is no landmark character between $c(o_1)$ and $c(o_2)$ whatsoever. Hence $p(o_1) = p(o_2)$ in state s . In this case, we break the tie by comparing their site ids. Without loss of generality, assume $\text{id}(o_1) < \text{id}(o_2)$. Then we have $c(o_1) \prec c(o_2)$ by definition of relation $\prec$. In get_it_er , we get the same result. $\square$

LEMMA 3. *$\text{get_it_er}(o_1, o_2)$ is correct if (1) $t(o_1) = t(o_2) = \text{del}$ and (2) $s = \text{est}(o_1) = \text{est}(o_2)$.*

Proof: Since o_1 and o_2 are to delete characters $c(o_1)$ and $c(o_2)$, respectively, that are both present in s , the relation between $s[c(o_1)]$ and $s[c(o_2)]$, or $p(o_1)$ and $p(o_2)$, can correctly determine the relation between $c(o_1)$ and $c(o_2)$. Therefore, if $p(o_1) < p(o_2)$, we have $c(o_1) \prec c(o_2)$; if $p(o_1) > p(o_2)$, we have $c(o_2) \prec c(o_1)$; and if $p(o_1) = p(o_2)$, we have $c(o_1) \equiv c(o_2)$. In get_it_er , we get the same results. $\square$

LEMMA 4. *$\text{get_it_er}(o_1, o_2)$ is correct if (1) $t(o_1) = \text{ins} \wedge t(o_2) = \text{del}$ and (2) $s = \text{est}(o_1) = \text{est}(o_2)$. The case of $t(o_1) = \text{del} \wedge t(o_2) = \text{ins}$ is symmetric.*

Proof: Similar to that of Lemma 3. The condition of $t(o_1) = \text{ins} \wedge t(o_2) = \text{del}$ means that o_1 is to insert a new character $c(o_1)$ into s and o_2 is to delete an existing character $c(o_2)$ in s . Hence the relation of $p(o_1)$ and $s[c(o_2)]$, or $p(o_2)$, determines the relation between $c(o_1)$ and $c(o_2)$. $\square$

COROLLARY 1. *The sufficient condition of $IT(o_1^s, o_2^s)$ is generally $s = \text{est}(o_1) = \text{est}(o_2)$. However, if $t(o_1) = t(o_2) = \text{ins}$ and $p(o_1) = p(o_2)$, it additionally requires $C_{ld}(c(o_1), c(o_2)) \subseteq s$, meaning that the landmark characters between $c(o_1)$ and $c(o_2)$, if any, must be present in s .*

4.2 ET and Preconditions

The purpose of $ET(o_2^s, o_1^{s^1})$, where $s^1 = \text{est}(o_1)$, $s^2 = \text{est}(o_2)$, and $s^2 = \text{exec}(s^1, o_1^{s^1})$, is to exclude the effect of $o_1^{s^1}$ from o_2^s such that the result $o_2^{s^1}$ can be correctly executed in s^1 , as if $o_1^{s^1}$ had not been executed earlier. As in Algorithm 3, we first call Algorithm 4 (get_et_er) to determine the effect relation between $o_1^{s^1}$ and o_2^s , and then use this relation to compute $o_2^{s^1}$.

Algorithm 3 $ET(o_2^s, o_1^{s^1})$: $o_2^{s^1}$

```

1:  $\text{get\_et\_er}(o_1, o_2)$ 
2: if  $c(o_2) \equiv c(o_1)$  then
3:   raise exception
4: else
5:    $o_2' \leftarrow o_2$ 
6:   if  $c(o_1) \prec c(o_2)$  then
7:     if  $t(o_1) = \text{ins}$  then
8:        $p(o_2') \leftarrow p(o_2) - 1$ 
9:     else  $\{ t(o_1) = \text{del} \}$ 
10:       $p(o_2') \leftarrow p(o_2) + 1$ 
11:   end if
12:   end if
13:   return  $o_2'$ 
14: end if

```

If $c(o_1)$ and $c(o_2)$ are the same, meaning that o_2 is to delete the character inserted by o_1 , or $o_1 \rightarrow o_2$ and $c(o_2) \equiv c(o_1)$. It does not make sense to pretend that o_1 had not been executed before o_2 due to causality. In this case we raise an exception. This corresponds to lines 13-14 in Algorithm 4 and lines 2-3 in Algorithm 3.

If $c(o_2) \prec c(o_1)$, meaning that $c(o_1)$ is on the right side of $c(o_2)$, then excluding the effect of o_1 does not have any impact on o_2 . Hence o_2 is returned as it is. If $c(o_1) \prec c(o_2)$, however, the position parameter of o_2 should be decreased

by one had o_1 not inserted $c(o_1)$ on its left; or the position parameter of o_2 should be increased by one had o_1 not deleted $c(o_1)$ on its left.

Algorithm 4 $\text{get_et_er}(o_1, o_2)$: relation of $c(o_1)$ and $c(o_2)$

```

1: return the relation if it is found in  $ER$ 
2: if  $p(o_1) < p(o_2)$  then
3:    $er \leftarrow c(o_1) \prec c(o_2)$ 
4: else if  $p(o_1) > p(o_2)$  then
5:    $er \leftarrow c(o_2) \prec c(o_1)$ 
6: else  $\{ p(o_1) = p(o_2) \}$ 
7:   if  $t(o_1) = t(o_2) = \text{ins}$  then
8:      $er \leftarrow c(o_2) \prec c(o_1)$ 
9:   else if  $t(o_1) = t(o_2) = \text{del}$  then
10:     $er \leftarrow c(o_1) \prec c(o_2)$ 
11:   else if  $t(o_1) = \text{del} \wedge t(o_2) = \text{ins}$  then
12:     $er \leftarrow c(o_2) \prec c(o_1)$ 
13:   else  $\{ t(o_1) = \text{ins} \wedge t(o_2) = \text{del} \}$ 
14:     $er \leftarrow c(o_1) \equiv c(o_2)$ 
15:   end if
16: end if
17: record  $er$  in  $ER$ 
18: return  $er$ 

```

In Algorithm 4, we define rules for determining the relation of $c(o_2)$ and $c(o_1)$ in ET only by their position parameters. Previous work [13, 14] proposed that the precondition of ET be $s^2 = \text{exec}(s^1, o_1)$. However, this condition alone is not sufficient for determining the relation of $c(o_1)$ and $c(o_2)$ in ET. There are counterexamples that show the relation can not be correctly determined even though this condition is satisfied [6]. In the following lemmas we examine the sufficient conditions of get_et_er by cases and then based on these cases we draw conclusions in Corollary 2. The sufficient conditions of ET are exactly the same as those of get_et_er . Hence the same conclusions hold.

LEMMA 5. $\text{get_et_er}(o_1, o_2)$ is correct if (1) $o_1 \parallel o_2$ and (2) $s^2 = \text{exec}(s^1, o_1)$.

Proof: If $o_1 \parallel o_2$, then o_2 must have included the effect of o_1 . That is, $o_2^2 = \text{IT}(o_2^1, o_1^1)$ must have been processed. Then relation $\prec$ between $c(o_1)$ and $c(o_2)$ has been correctly determined in get_it_er or IT. If relation $\prec$ is recorded and retrieved, get_et_er can be correctly processed. $\square$

LEMMA 6. $\text{get_et_er}(o_1, o_2)$ is correct if (1) $t(o_1) = t(o_2) = \text{ins}$, (2) $o_1 \rightarrow o_2$, and (3) $s^2 = \text{exec}(s^1, o_1)$.

Proof: By the given conditions, we know that $c(o_1) \in s^2$ must be true. Since two insertions cannot insert the same character, it is impossible to have $c(o_1) \equiv c(o_2)$. If o_2 inserts on the right of $c(o_1)$ in s^2 , it must be $p(o_2) > p(o_1)$ and $c(o_1) \prec c(o_2)$. If o_2 inserts on the left of $c(o_1)$ in s^2 , or $p(o_2) \leq p(o_1)$ by the semantics of insertion, it must be $c(o_2) \prec c(o_1)$. From get_et_er , we get the same results. $\square$

LEMMA 7. $\text{get_et_er}(o_1, o_2)$ is correct if (1) $t(o_1) = t(o_2) = \text{del}$, (2) $o_1 \rightarrow o_2$, and (3) $s^2 = \text{exec}(s^1, o_1)$.

Proof: By the given conditions, $c(o_1)$ must appear in s^1 and $p(o_1) = s^1[c(o_1)]$; $c(o_2)$ must appear in both s^1 and s^2 and $p(o_2) = s^2[c(o_2)]$. Since o_2 cannot delete the same character as o_1 , it is impossible to have $c(o_1) \equiv c(o_2)$.

If $p(o_1) \leq p(o_2)$, we have $c(o_1) \prec c(o_2)$ by get_et_er . Assume to the contrary $c(o_2) \prec c(o_1)$. Then $s^1[c(o_2)] < s^1[c(o_1)] = p(o_1)$ and $s^2[c(o_2)] = s^1[c(o_2)]$. That is, $s^1[c(o_2)] = p(o_2)$ and $p(o_2) < p(o_1)$, contradicting the precondition.

On the other hand, if $p(o_2) < p(o_1)$, we have $c(o_2) \prec c(o_1)$ by get_et_er . Assume to the contrary $c(o_1) \prec c(o_2)$. Then $s^1[c(o_1)] < s^1[c(o_2)]$ and $s^2[c(o_2)] = s^1[c(o_2)] - 1$. That is, $s^1[c(o_2)] = s^2[c(o_2)] + 1$ and $p(o_1) < p(o_2) + 1$. Hence $p(o_1) \leq p(o_2)$, which contradicts $p(o_2) < p(o_1)$. $\square$

LEMMA 8. $\text{get_et_er}(o_1, o_2)$ is correct if (1) $t(o_1) = \text{ins} \wedge t(o_2) = \text{del}$, (2) $o_1 \rightarrow o_2$, and (3) $s^2 = \text{exec}(s^1, o_1)$.

Proof: Similar to that of Lemma 6. By the given conditions, $c(o_1)$ must appear in s^2 and $p(o_1) = s^2[c(o_1)]$. Since o_2 is to delete a character in s^2 , it must be that $p(o_2) = s^2[c(o_2)]$. Hence by comparing $p(o_1)$ and $p(o_2)$ we can unambiguously determine the effect relation of o_1 and o_2 . $\square$

LEMMA 9. $\text{get_et_er}(o_1, o_2)$ is correct if (1) $t(o_1) = \text{del} \wedge t(o_2) = \text{ins}$, (2) $o_1 \rightarrow o_2$, (3) $s^2 = \text{exec}(s^1, o_1)$, and (4) $C_{ld}(c(o_1), c(o_2)) \subseteq s^2$.

Proof: Since o_2 inserts a new character in s^2 after o_1 deletes an existing one from s^1 , it is impossible to have $c(o_1) \equiv c(o_2)$. If $p(o_1) = p(o_2)$ and $C_{ld}(c(o_1), c(o_2)) \subseteq s^2$, we know that $c(o_1) \notin \text{est}(o_2)$ and there is no landmark character between $p(o_1)$ and $p(o_2)$ in s^2 . Hence we mandate $c(o_2) \prec c(o_1)$ by the tie-breaking policy in the definition of relation $\prec$. However, if $p(o_1) \neq p(o_2)$, there must be at least one landmark character between $p(o_1)$ and $p(o_2)$ in s^2 . Hence $c(o_1) \prec c(o_2)$ if $p(o_1) < p(o_2)$, and $c(o_2) \prec c(o_1)$ if $p(o_1) > p(o_2)$. We draw the same conclusions from $\text{get_et_er}()$. $\square$

However, if (1) $t(o_1) = \text{del}$ and $t(o_2) = \text{ins}$, (2) $o_1 \rightarrow o_2$, and (3) $s^2 = \text{exec}(s^1, o_1)$, but (4) $C_{ld}(c(o_1), c(o_2)) \not\subseteq s^2$, it has been shown in [6] that the rules in get_et_er cannot correctly determine the effect relation of o_1 and o_2 . We call this condition the **unsafe condition** of ET.

COROLLARY 2. The sufficient condition of $\text{ET}(o_1^2, o_2^1)$ is generally $s^2 = \text{est}(o_2) = \text{exec}(s^1, o_1)$. However, if $o_1 \rightarrow o_2$ and $p(o_1) = p(o_2)$ and $t(o_1) = \text{del}$ and $t(o_2) = \text{ins}$, it additionally requires $C_{ld}(c(o_1), c(o_2)) \subseteq s^2$, meaning that the landmark characters between $c(o_1)$ and $c(o_2)$, if any, must be present in s^2 .

5. A CONTROL ALGORITHM

To integrate a remote operation o , we often have to transform o against a sequence of operations sq . We denote the process of inclusively transforming o against sq as $\text{ITSQ}(o, sq)$ and that of exclusively transforming o against sq as $\text{ETSQ}(o, sq)$. The above-defined preconditions of IT and ET may not always hold at each step. This section discusses how to solve this problem. We first discuss ETSQ in Section 5.1 and then ITSQ in Section 5.2. The control procedure is explained in Section 5.3. Finally Section 5.4 proves correctness.

Two operations o_1 and o_2 are contextually serialized iff $\text{exec}(\text{est}(o_1), o_1) = \text{est}(o_2)$. A sequence of operations $o_1, o_2, \dots, o_n$ is contextually serialized iff for any i , $1 \leq i \leq n-1$, $\text{exec}(\text{est}(o_i), o_i) = \text{est}(o_{i+1})$. For brevity, we often refer to a contextually serialized sequence as a sequence.

We generalize function $s' = \text{exec}(s, o)$ to sequences: Given state s^0 and sequence sq , where $s^0 = \text{est}(sq[0])$ and $|sq| = n$, $s^n = \text{exec}(s^0, sq) = \text{exec}(\text{exec}(s^0, sq[0]), sq[1, n-1])$. That

is, executing a sequence sq in state s^0 is equivalent to executing $sq[0]$ in s^0 , which results in $s^1 = \text{exec}(s^0, sq[0])$, then executing $sq[1]$ in $s^1 = \text{exec}(s^1, sq[1])$, and so forth.

Given two operation sequences sq_1 and sq_2 , if $\text{est}(sq_2[0]) = \text{exec}(\text{est}(sq_1[0]), sq_1)$, we also say that sq_1 and sq_2 are contextually serialized. For the same reason, operation $sq_1[n-1]$ and sequence sq_2 , sequence sq_1 and operation $sq_2[0]$, are also contextually serialized. Notation $sq_1 + sq_2$ concatenates two sequences sq_1, sq_2 into one sequence.

5.1 ETSQ

ETSQ(o, sq) requires that sq and o be contextually serialized, i.e., $\text{est}(o) = \text{exec}(\text{est}(sq[0]), sq)$. Hence to compute ETSQ(o, sq), in principle we should exclusively transform o against $sq[n-1], sq[n-2], \dots, sq[0]$ in turn. Obviously the key is how to ensure the correctness of each step under the unsafe condition of ET. We address this problem by constructing an effects-equivalent special ET-safe sequence $etsos$ out of sq such that $\text{exec}(\text{est}(sq[0]), sq) \equiv \text{exec}(\text{est}(sq[0]), etsos)$, and then perform ETSQ($o, etsos$) instead.

DEFINITION. 3. A given sequence sq is an ET-Safe Operation Sequence (ETSOS) if for any i, j , where $0 \leq i < j \leq n-1$ and $|sq| = n$, either (1) $c(sq[i]) \prec c(sq[j])$ or (2) $c(sq[i]) \equiv c(sq[j])$ and $t(sq[i]) = \text{ins}$ and $t(sq[j]) = \text{del}$.

An ETSOS sq is interesting due to the following two properties stated in Lemmas 10 and 11. In particular, we can infer the relation between $c(o)$ and effects of all operations in $sq[0, i-1]$ if the relation between $c(o)$ and some $c(sq[i])$ is known, where $0 \leq i \leq |sq| - 1$.

LEMMA 10. Given operation o and an ETSOS sq , if there exists some $sq[i]$, $0 \leq i \leq |sq| - 1$, such that $c(sq[i]) \prec c(o)$, then $c(sq[j]) \prec c(o)$ for all j : $0 \leq j \leq i - 1$.

The above lemma actually assumes the correctness of ET($o, sq[i]$) for $0 \leq i < |sq|$. The following lemma establishes under which conditions this can be assumed. We introduce notation C_{sq} to denote the set of characters that appear in $\text{est}(sq[0])$ and sq . That is, $C_{sq} = \bigcup_{i=0}^{n-1} \{c(sq[i])\} \cup \text{est}(sq[0])$.

LEMMA 11. Given operation o and sequence sq , where $|sq| = n$ and $\text{est}(o) = \text{exec}(\text{est}(sq[0]), sq)$, then ETSQ(o, sq) is correct if (1) sq is an ETSOS, and (2) $C_{id}(c(o), c(sq[i])) \subseteq C_{sq}$ for all $sq[i] \rightarrow o$.

Proof: We only need to prove the correctness of ET($o, sq[i]$) for any $sq[i]$ under the unsafe conditions. That is, $p(sq[i]) = p(o)$, $sq[i] \rightarrow o$, $t(sq[i]) = \text{del}$, and $t(o) = \text{ins}$. We prove this by induction.

Basis case: consider ET($o, sq[n-1]$). By definition of ETSOS, we know $c(sq[k]) \prec c(sq[n-1])$ for $0 \leq k < n-1$. By condition $C_{id}(c(o), c(sq[n-1])) \subseteq C_{sq}$, we know $\nexists c: c(sq[n-1]) \prec c \prec c(o)$ for otherwise we would have got $p(sq[n-1]) < p(o)$, which contradicts $p(sq[n-1]) = p(o)$. Then by the definition of relation $\prec$, whether or not $\exists c: c(o) \prec c \prec c(sq[n-1])$, we get $c(o) \prec c(sq[n-1])$. Hence ET($o, sq[n-1]$) is correct.

Induction: Assume that processing of ETSQ($o, sq[i+1, n-1]$) is correct. Let o' be the execution form of o relative to $\text{est}(sq[i+1])$. By Lemma 10, doing ET($o', sq[i]$) means that $c(o) \prec c(sq[k])$ where $i+1 \leq k \leq n-1$. Because $C_{id}(c(o), c(sq[i])) \subseteq C_{sq}$, we have $\nexists c: c(sq[i]) \prec c \prec c(o)$

for otherwise we would have got $p(sq[i]) < p(o)$, which contradicts $p(sq[i]) = p(o)$. Then by the definition of relation $\prec$, whether or not $\exists c: c(o) \prec c \prec c(sq[i])$, we get $c(o) \prec c(sq[i])$. Hence ET($o, sq[i]$) is correct. $\square$

Algorithm 5 ETSQ(o, sq): o'

```

1:  $o' \leftarrow o$ 
2: for  $i = |sq| - 1$  to 0 step -1 do
3:    $o' \leftarrow \text{ET}(o', sq[i])$ 
4:   if  $c(sq[i]) \prec c(o')$  then
5:     break
6:   end if
7: end for
8: for  $j = i - 1$  to 0 step -1 do
9:   if  $t(sq[j]) = \text{ins}$  then
10:     $p(o') = p(o') - 1$ 
11:   else
12:     $p(o') = p(o') + 1$ 
13:   end if
14: end for
15: return  $o'$ 

```

Based on Lemmas 10 and 11, we define ETSQ(o, sq) in Algorithm 5. The precondition is given in Lemma 11. The function exclusively transforms o with operations in sq from $sq[n-1]$ to $sq[0]$. When the condition defined in Lemma 10 is satisfied, we break the loop of lines 2-7 because it is no longer necessary to call ET() on the remaining operations in sq . Instead, effects of those operations are excluded directly as in lines 8-14.

5.2 ITSQ

The basic idea of ensuring the correctness of ITSQ is similar to that of ETSQ. We rebuild a special operation sequence $itsos$ from a given sequence sq such that ITSQ($o, itsos$) achieves the same effects as ITSQ(o, sq).

DEFINITION. 4. A sequence sq is an IT-Safe Operation Sequence (ITSOS) iff for any two operations $sq[i]$ and $sq[j]$, if $t(sq[i]) = \text{ins}$ and $t(sq[j]) = \text{del}$, then $i < j$ holds.

An ITSOS sq can be rewritten as $sq_i + sq_d$, where sq_i includes all insertions in sq and sq_d all deletions in sq .

LEMMA 12. Given operation o and sequence sq , ITSQ(o, sq) is correct, if $\text{est}(sq[0]) = \text{est}(o)$ and either (1) $t(o) = \text{del}$ or (2) $t(o) = \text{ins} \wedge sq$ is an ITSOS $\wedge$ for any insertion $sq[i]$: $C_{id}(c(sq[i]), c(o)) \subseteq \text{est}(sq[i])$.

Proof: In case $t(o) = \text{del}$, by Corollary 1, ITSQ($o, sq[i]$) is correct for any operation $sq[i]$. Consider case $t(o) = \text{ins}$. Because sq is an ITSOS, any insertion is processed before any deletion. That is, inclusively transforming o against any insertion does not lose (delete) any landmark character. Because $C_{id}(c(sq[i]), c(o)) \subseteq \text{est}(sq[i])$, due to Corollary 1, IT($o, sq[i]$) is correct for any insertion $t(sq[i])$ and IT($o, sq[j]$) is correct for any deletion $sq[j]$. $\square$

Based on the precondition given in Lemma 12, we define ITSQ(o, sq) in Algorithm 6, which is self-explained.

5.3 The Integration Procedure

The top-level control algorithm executes local and remote operations. An operation log (or history buffer HB) is maintained at each site. For best responsiveness, any local operation is executed and appended to HB once it is generated.

Algorithm 6 ITSQ(o, sq): o'

```
1:  $o' \leftarrow o$ ;
2: for  $i = 0$  to  $|sq|-1$  do
3:    $o' \leftarrow \text{IT}(o', sq[i])$ 
4:   if  $o' = \phi$  then
5:     break
6:   end if
7: end for
8: return  $o'$ 
```

Every remote operation o is queued until it is causally-ready, i.e., all operations that happened before o are in HB . Suppose the initial state is s^0 and the current state s^n , where $n = |HB|$. Then function $\text{Integrate}(o, HB)$ is called to derive its execution form o' relative to s^n . If $o' = \phi$, it is discarded. Otherwise o' is executed and appended to HB . Apparently HB is a contextually serialized sequence.

Algorithm 7 Integrate(o, HB): o'

```
1:  $\langle sq_h, sq_c \rangle \leftarrow \text{TransposePrecCon}(o, HB)$ 
2: if  $sq_c = []$  then
3:   return  $o' \leftarrow o$ 
4: end if
5: if  $t(o) = \text{del}$  then
6:   return  $o' \leftarrow \text{ITSQ}(o, sq_c)$ 
7: else {  $t(o) = \text{ins}$  }
8:    $sq'_h \leftarrow \text{BuildETSOS}(sq_h)$ 
9:    $\langle sq_{hi}, sq_{hd} \rangle \leftarrow \text{TransposeInsDel}(sq'_h)$ 
10:   $o' \leftarrow \text{ETSQ}(o, sq_{hd})$ 
11:   $sq \leftarrow \text{BuildETSOS}(sq_{hd} + sq_c)$ 
12:   $\langle sq_i, sq_d \rangle \leftarrow \text{TransposeInsDel}(sq)$ 
13:  return  $o' \leftarrow \text{ITSQ}(o', sq_i + sq_d)$ 
14: end if
```

In $\text{Integrate}(o, HB)$, we first call $\text{TransposePrecCon}(o, HB)$ to transpose HB into two sequences: sq_h , which includes all operations that happened before o , and sq_c , which includes all operations that are concurrent with o . Due to causality preservation, we have $\text{gst}(o) = \text{exec}(s^0, sq_h)$.

If sq_c is empty, it means $s^n = \text{gst}(o)$. Then o is returned as it is (lines 2-3) and will be executed in s^n directly. If sq_c contains concurrent operations, we must incorporate the effects of those operations into o to get its execution form o' in s^n . By the above analysis, we cannot perform $\text{ITSQ}(o, sq_c)$ until the preconditions defined in Lemma 12 are satisfied. If $t(o) = \text{del}$, $\text{ITSQ}(o, sq_c)$ is guaranteed to be correct (lines 5-6). However, if $t(o) = \text{ins}$, we must construct some ITSOS sq' such that $C_{ld}(c(o), sq'[i]) \subseteq \text{est}(o)$ for every insertion $sq'[i]$. Then $\text{IT}(o, sq'[i])$ is always correct and the problem can be translated to $\text{ITSQ}(o, sq')$. This is achieved as follows.

We first call function $\text{TransposeInsDel}(sq_h)$ to transpose sequence sq_h into two subsequences, sq_{hi} , which contains all insertions, and sq_{hd} , which contains all deletions, and sq_{hi} and sq_{hd} are contextually serialized. After that, we call $\text{ETSQ}(o, sq_{hd})$ to exclude the effects of sq_{hd} from o . Now o is defined in state $s^m = \text{exec}(s^0, sq_{hi})$, where $m = |sq_{hi}|$. After that, we call $\text{TransposeInsDel}(sq)$ to get $sq' = sq_i + sq_d$. Operation $sq_i[0]$ is also defined in state s^m . Since sq_{hi} only contains insertions, all the landmark characters between $c(o)$ and any $sq_i[k]$ must be included in state s^m . Therefore $\text{ITSQ}(o, sq')$ can be correctly processed.

Algorithm 8 TransposePrecCon(o, sq): $\langle sq_h, sq_c \rangle$

```
1:  $sq_h \leftarrow []$ ;  $sq_c \leftarrow []$ 
2: for  $i=0$  to  $|sq'|-1$  do
3:   if  $sq[i] \parallel o$  then
4:      $sq_c \leftarrow sq_c + sq[i]$ 
5:   else
6:      $\langle o_i, sq_c \rangle \leftarrow \text{TransposeOSq}(sq_c, sq[i])$ 
7:      $sq_h \leftarrow sq_h + o_i$ 
8:   end if
9: end for
10: return  $\langle sq_h, sq_c \rangle$ 
```

Algorithm 9 TransposeInsDel(sq): $\langle sq_i, sq_d \rangle$

```
1:  $sq_i \leftarrow []$ ;  $sq_d \leftarrow []$ 
2: for  $i=0$  to  $|sq|-1$  do
3:   if  $t(sq[i]) = \text{del}$  then
4:      $sq_d \leftarrow sq_d + sq[i]$ 
5:   else
6:      $\langle o_i, sq_d \rangle \leftarrow \text{TransposeOSq}(sq_d, sq[i])$ 
7:      $sq_i \leftarrow sq_i + o_i$ 
8:   end if
9: end for
10: return  $\langle sq_i, sq_d \rangle$ 
```

The four transposition functions 8, 9, 10, 11 are straightforward. Function $\text{TransposePrecCon}()$ is adopted from [11, 13]. It transposes sq into two subsequences sq_h and sq_c , such that sq_h , all operations that happened before o , are contextually serialized before sq_c , all those that are concurrent with o . Lemma 13 will show that IT/ET in $\text{TransposePrecCon}()$ only happen between concurrent operations.

$\text{TransposeInsDel}(sq)$ transposes sq such that all insertions are contextually serialized before all deletions. The precondition of $\text{TransposeOSq}(sq, o)$ is that sq is contextually serialized before o . The output is such that o' is contextually serialized before sq' . $\text{Transpose}(o_2, o_1)$ is to transpose two contextually serialized operations o_2 and o_1 such that in the output o'_1 is contextually serialized before o'_2 . Since o_2 is contextually serialized before o_1 , we first do $o'_1 = \text{ET}(o_1, o_2)$ such that o'_1 and o_2 are defined in the same state, and then do $o'_2 = \text{IT}(o_2, o'_1)$ such that o'_2 is defined in state $\text{exec}(\text{est}(o'_1), o'_1)$. Note if o_1 depends on o_2 , i.e., $t(o_2) = \text{ins}$, $t(o_1) = \text{del}$, $o_2 \rightarrow o_1$, and $p(o_1) = p(o_2)$, then we know $c(o_1) \equiv c(o_2)$ and return $\langle o_1, o_2 \rangle$. However, this case does not appear in TransposePreCon and TransposeInsDel but may appear in BuildETSOS .

Function $\text{BuildETSOS}(sq)$ incrementally builds an ET-SOS sq' . Initially sq' only includes $sq[0]$. Then each time a new element $sq[i]$ is added into sq' . It transposes $sq[i]$ with operations in sq' from right to left until some $sq'[j]$ is found (line 13) such that $c(sq'[j]) \prec c(o)$ or $c(sq'[j]) \equiv c(o)$, which is exactly the stop condition defined in Lemma 10. Under this case, o is inserted after $sq'[j]$ (line 14). Then the relation between $c(o)$ and other operations in the rest of sq is known and recorded (line 10). If the stop condition never appears, as in lines 19-20, then o is added to the head of sq' .

Note the use of function $\text{BuildETSOS}()$ in lines 8 and 11 of $\text{Integrate}()$. $\text{BuildETSOS}(sq_h)$ in line 8 is to ensure the correct processing of lines 9-10. It can be shown that sq_h satisfies the precondition required in Lemma 11 and hence $\text{BuildETSOS}(sq_h)$ is correct. It can also be shown

Algorithm 10 TransposeOSq(sq, o): $\langle o', sq' \rangle$

```
1:  $o' \leftarrow o$ ;  $sq' \leftarrow sq$ 
2: for  $i = |sq|-1$  to 0 step -1 do
3:    $\langle o', sq'[i] \rangle \leftarrow \text{Transpose}(sq'[i], o')$ 
4: end for
5: return  $\langle o', sq' \rangle$ 
```

Algorithm 11 Transpose(o_2, o_1): $\langle o'_1, o'_2 \rangle$

```
1: if  $c(o_1) \equiv c(o_2)$  then {determined as in get_et_er}
2:   return  $\langle o_1, o_2 \rangle$ 
3: else
4:    $o'_1 \leftarrow \text{ET}(o_1, o_2)$ 
5:    $o'_2 \leftarrow \text{IT}(o_2, o'_1)$ 
6:   return  $\langle o'_1, o'_2 \rangle$ 
7: end if
```

that IT/ET in line 9 is correct and that the conditions of Lemma 11 are satisfied in line 10. Similarly, BuildETSOS in line 11 ensures the correctness of line 12.

5.4 Correctness Proofs

LEMMA 13. *TransposePrecCon() is correct.*

Proof: Due to definition of this function, whenever two operations $sq[i]$ and $sq[i+1]$ are transposed, we always have $sq[i] \parallel sq[i+1]$, which has been proved in [12]. Then $sq[i+1]$ must have been inclusively transformed with $sq[i]$ before and their relation is already known. Therefore Transpose() can always be correctly processed. It is easy to see that the two resulted sequences sq_h and sq_c are also causality-preserving since they keep the same order of operations as in sq . $\square$

LEMMA 14. *BuildETSOS() is correct.*

Proof: The process of building an ETSOS sq' from input sequence sq resembles that of insertion sort. Each time we add a new operation $sq[i+1]$ into the partial result sq' that was built from $sq[0, i]$. Sequence sq' is scanned from right to left to find out the right position to insert $sq[i+1]$. Note that each operation in sequence sq' actually represents a state transition. Hence, to insert $sq[i+1]$ before some position $sq'[j]$, we have to *transpose* $sq[i+1]$ and $sq'[j, i]$ by calling function Transpose(); That is, the correctness of BuildETSOS() depends on the correctness of Transpose().

In Transpose(o_1, o_2), ET is called before IT. If $\text{ET}(o_1, o_2)$ is correct, then the relation of $c(o_1)$ and $c(o_2)$ is known, which ensures the correctness of IT. Hence to prove the correctness of BuildETSOS(), we only need to prove the correctness of $\text{ETSQ}(sq[i+1], sq'[j, i])$. This can be further reduced to proving that the conditions of Lemma 11 can be satisfied before calling BuildETSOS() in lines 8 and 11 in Integrate(). The proofs of these two cases are similar. For space reasons we only prove the case of line 8. In this case, the input sequence is sq_h , which is causality-preserving due to the proof of Lemma 13. We prove the correctness of $sq'_h = \text{BuildETSOS}(sq_h)$ by induction.

Base: $\text{ET}(sq_h[1], sq_h[0])$. If $sq_h[1] \parallel sq_h[0]$, then $\text{ET}(sq_h[1], sq_h[0])$ is correct by Lemma 5. Otherwise, it must be $sq_h[0] \rightarrow sq_h[1]$. Because $\text{gst}(sq_h[0]) \equiv s^0$ and there is no other operation, we have $C_{ld}(c(sq_h[0]), c(sq_h[1])) \subseteq \text{est}(sq_h[1])$. Then $\text{ET}(sq_h[1], sq_h[0])$ is correct.

Algorithm 12 BuildETSOS(sq): sq'

```
1: if  $|sq| < 1$  then
2:   return  $sq' \leftarrow sq$ 
3: end if
4:  $sq' \leftarrow [sq[0]]$ 
5: for  $i=1$  to  $|sq|-1$  do
6:    $o \leftarrow sq[i]$ 
7:    $flag \leftarrow \text{false}$ 
8:   for  $j = |sq'|-1$  to 0 step -1 do
9:     if  $flag = \text{true}$  then
10:      store  $c(sq'[j]) \prec c(o)$ 
11:     else
12:        $\langle o, sq'[j] \rangle \leftarrow \text{Transpose}(sq'[j], o)$ 
13:       if  $c(sq'[j]) \prec c(o)$  or  $c(sq'[j]) \equiv c(o)$  then
14:          $sq' \leftarrow sq'[0, j] + o + sq'[j+1, |sq'|-1]$ 
15:          $flag \leftarrow \text{true}$ 
16:       end if
17:     end if
18:   end for
19:   if  $flag = \text{false}$  then
20:      $sq' \leftarrow o + sq'$ 
21:   end if
22: end for
23: return  $sq'$ 
```

Induction: Suppose sq'_h is the ETSOS correctly built from $sq_h[0, i]$. Now we examine how $sq_h[i+1]$ is added into sq'_h . Let $C_{sq'_h} = \bigcup_{j=0}^i (c(sq'_h[j]))$. Since $sq'_h[0]$ is defined on s^0 and all operations that happened before $sq_h[i+1]$ are in sq'_h , it is obvious that $C_{ld}(c(sq_h[i+1]), c(sq'_h[j])) \subseteq C_{sq'_h}$ for any $0 \leq j < |sq'_h|$ and $sq'_h[j] \rightarrow sq_h[i+1]$. By Lemma 11, $\text{ETSQ}(sq_h[i+1], sq'_h)$ is correct. $\square$

LEMMA 15. *TransposeInsDel() is correct.*

Proof: Function TransposeInsDel() is called twice in Function Integrate(), lines 9 and 12, respectively. In either place, the input sequence of TransposeInsDel() goes through function BuildETSOS() such that relation $\prec$ between any two operations in the input sequence is known. Hence IT/ET in TransposeInsDel() is always correct, which ensures the correctness of TransposeInsDel(). $\square$

LEMMA 16. *Integrate() is correct.*

Proof: Due to Lemma 13, $\text{TransposePrecCon}(o, HB)$ correctly transposes HB into sq_h and sq_c . Sequence sq_h contains all the operations that happened before o and preserves their happened-before relation. Due to causality preservation, $\text{gst}(o) = \text{exec}(s^0, sq_h) = \text{est}(sq_c[0])$.

We would have directly performed $o' = \text{ITSQ}(o, sq_c)$ to include the effects of sq_c into o' . However, it is possible that sq_h contains deletions such that $\text{est}(sq_c[0])$ does not contain $C_{ld}(c(o), c(sq_c[0]))$ and the correctness of $\text{IT}(o, sq_c[0])$ cannot be guaranteed. To solve this problem, we first transpose sq_h into sq_{hi} and sq_{hd} such that sq_{hi} contains all insertions that happened before o and sq_{hd} includes all deletions that happened before o . Due to Lemmas 14 and 15, this step is correct. It is easy to see from the process of TransposeInsDel() that sq_{hd} is an ETSOS. In addition, let $C_{sq_{hd}} = \text{est}(sq_{hd}[0]) \bigcup_{j=0}^{|sq_{hd}|-1} sq_{hd}[j] = \text{exec}(s^0, sq_{hi}) \bigcup_{j=0}^{|sq_{hd}|-1} sq_{hd}[j]$. Since all possible landmark characters between o and $sq_{hd}[i]$, if any,

are introduced by sq_{hi} , we conclude that sequence sq_{hd} satisfies the preconditions of Lemma 11. Hence processing $o' = \text{ETSQ}(o, sq_{hd})$ correctly transforms o into o' , which is defined relative to state $s' = \text{est}(sq_{hd}[0]) = \text{exec}(s^0, sq_{hi})$.

Next we transpose sequence $sq = sq_{hd} + sq_c$ into $sq' = sq_i + sq_d$ such that sq_i includes all the insertions and sq_d includes all the deletions in sq . Due to Lemmas 15, this step is correct and sq' is an ITSOS. Because $sq_{hi} + sq_i$ does not contain any deletions, the preconditions of Lemma 12 are satisfied and hence $o'' = \text{ITSQ}(o', sq')$ is correct. $\square$

6. AN EXAMPLE

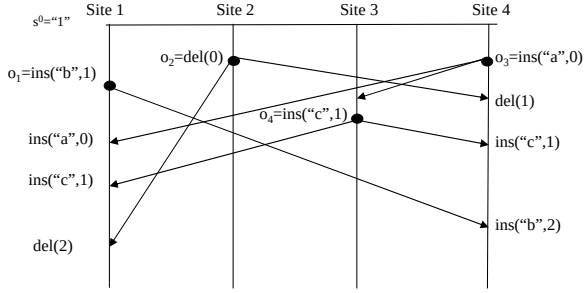


Figure 2: An example: four sites start from state “1” and converge in “acb”.

Figure 2 shows a group editing session with four sites. Assume that the initial state is $s^0 = “1”$. Site 1 performs an operation $o_1 = \text{ins}(“b”, 1)$. Concurrently, site 2 performs $o_2 = \text{del}(0)$, and site 4 performs $o_3 = \text{ins}(“a”, 0)$. After o_3 is executed, site 3 performs $o_4 = \text{ins}(“c”, 1)$. We have $o_1 \parallel o_2$, $o_2 \parallel o_3$, $o_1 \parallel o_3$, $o_3 \rightarrow o_4$, $o_1 \parallel o_4$, and $o_2 \parallel o_4$. For simplicity, we only consider how these operations are executed at site 1 and site 3, assuming that the execution order at site 1 is o_1, o_3, o_4, o_2 , and the order at site 4 is o_3, o_2, o_4, o_1 .

Site 1: Let $s_1^1 = \text{exec}(s^0, o_1) = “1b”$. Due to $o_3 \parallel o_1$, we have $o_3^{s_1^1} = \text{IT}(o_3, o_1) = \text{ins}(“a”, 0)$. Executing $o_3^{s_1^1}$ in s_1^1 results in $s_1^2 = \text{exec}(s_1^1, o_3^{s_1^1}) = “a1b”$. When o_4 arrives, the history buffer of site 1 is $[o_1, o_3^{s_1^1}]$. According to the control algorithm, we transpose it into $sq_h = [o_3]$ and $sq_c = [o_1^{s_1^1}]$, where $s_1^{s_1^1} = \text{exec}(s^0, o_3) = “a1”$ and $o_1^{s_1^1} = \text{IT}(o_1, o_3) = \text{ins}(“b”, 2)$. Inclusively transforming o_4 against sq_c results in $o_4^{s_1^2} = \text{ITSQ}(o_4, sq_c) = \text{ins}(“c”, 1)$, the execution of which leads to $s_1^3 = \text{exec}(s_1^2, o_4^{s_1^2}) = “ac1b”$. Finally, when o_2 arrives, the history buffer is $HB = [o_1, o_3^{s_1^1}, o_4^{s_1^2}]$. Since all these operations are concurrent with o_2 , we get $o_2^{s_1^3} = \text{ITSQ}(o_2, HB) = \text{del}(2)$ and $s_1^4 = \text{exec}(s_1^3, o_2^{s_1^3}) = “acb”$.

Site 4: $s_4^1 = \text{exec}(s^0, o_3) = “a1”$. When o_2 arrives, we compute $o_2^{s_4^1} = \text{IT}(o_2, o_3) = \text{del}(1)$. After executing $o_2^{s_4^1}$, we get state $s_4^2 = “a”$. When o_4 arrives, $o_4^{s_4^2} = \text{IT}(o_4, o_2^{s_4^1}) = \text{ins}(“c”, 1)$, the execution of which results in $s_4^3 = “ac”$. Finally, o_1 arrives, the history buffer of site 4 is $HB = [o_3, o_2^{s_4^1}, o_4^{s_4^2}]$. Since all these operations are concurrent with o_1 , we first call function $\text{BuildETSOS}()$ to convert HB into an ETSOS, $sq = [o_3, o_4^{s_4^2}, o_2^{s_4^1}]$, where $o_4^{s_4^2} = \text{ins}(“c”, 1)$ and $o_2^{s_4^1} = \text{del}(2)$. Then we call function $\text{TransposeInsDel}()$ to convert sq into $sq_i = [o_3, o_4^{s_4^2}]$ and $sq_d = [o_2^{s_4^1}]$. Let $sq' = sq_i + sq_d$.

We have $o_1^{s_4^3} = \text{ITSQ}(o_1, sq' = \text{ins}(“b”, 2))$. Hence $s_4^4 = \text{exec}(s_4^3, o_1^{s_4^3}) = “acb”$.

Similarly it can be shown that all the four sites converge in the same final state $s^4 = “acb”$. We can verify the correctness of s^4 by definitions in Section 3. Since o_1 is generated in state s^0 , we infer $‘1’ \prec ‘b’$ by definition. Similarly we can infer $‘a’ \prec ‘1’$ from the generation of o_3 , and $‘a’ \prec ‘c’ \prec ‘1’$ from the generation of o_4 . Globally we infer the total order $‘a’ \prec ‘c’ \prec ‘1’ \prec ‘b’$. Since o_2 is intended to delete character $‘1’$ at site 2, the correct result must be $“acb”$, which is s^4 .

7. RELATED WORK

Previous work [11, 14] has established that an OT algorithm is correct if it satisfies three criteria: causality preservation, convergence, and intention preservation. The intention preservation condition is the first attempt in the literature to constrain convergence in interactive groupware applications. Due to its intuitiveness, this model has been well-accepted in group editors, such as [10, 11, 13, 14, 15]. However, the intention preservation condition is not well-formalized and difficult to verify in practice. Even in its original definition [14], there is a well-known scenario identified as being intention-preserving but it is actually not [6].

Ressel et al [9] proves that any OT-based algorithm can achieve convergence in presence of arbitrary transformation paths if its IT function can satisfy two transformation properties that are called TP1 and TP2. However, these two properties are only for IT, because they were established before the concept of ET was proposed [11, 14].

TP1/TP2 are required in previous OT algorithms adOPTed [9], SOCT2 [11], and GOTO [13] but have never been verified. SDT [6] is the first that has been formally proved to satisfy TP2. However, the proof as presented in [4] only considers two primitive characterwise insert and delete operations. Due to the huge number of cases to consider, the proof scales poorly to a more sophisticated operation set.

Due to the widely-recognized difficulties in verifying TP2, a number of previous OT algorithms attempted to free TP2, such as GOT [14], SOCT3/4 [15], NICE [10], and TIBOT [8]. However, counterexamples show that, although they can achieve convergence, these algorithms fail to preserve the operation effects relation [4, 6].

It is proposed that $\text{IT}(o_1^s, o_2^s)$ must ensure $s = \text{est}(o_1) = \text{est}(o_2)$ and $\text{ET}(o_2^s, o_1^{s_1})$ must ensure $s^2 = \text{exec}(s^1, o_1^{s_1})$ to verify intention preservation [13, 14]. However, counterexamples show that these two conditions are not really sufficient [6]. Corollaries 1 and 2 in this paper establish sufficient conditions for IT and ET that are based on the notion of operation effects relation instead of intention preservation.

LBT improves SDT [6] in the following two ways. In Section 5, the execution time of function $\text{Integrate}()$ is dominated by functions $\text{TransposePrecCon}()$, $\text{BuildETSOS}()$, and $\text{TransposeInsDel}()$, each taking time quadratic to the size of the input sequence in the worst case. Hence the worst-case time complexity of LBT is $O(n^2)$, where n is the number of the history buffer (operation log). This outperforms SDT, the worst-case time complexity of which is $O(n^3)$.

To be more concrete, Figure 3 shows the results of some simple experiments we did to compare the worst-case performance of SDT and LBT. The algorithms were implemented in C++, compiled by GNU g++ v3.3.2, and executed on a computer running RedHat Linux kernel 2.4.22 with an Intel

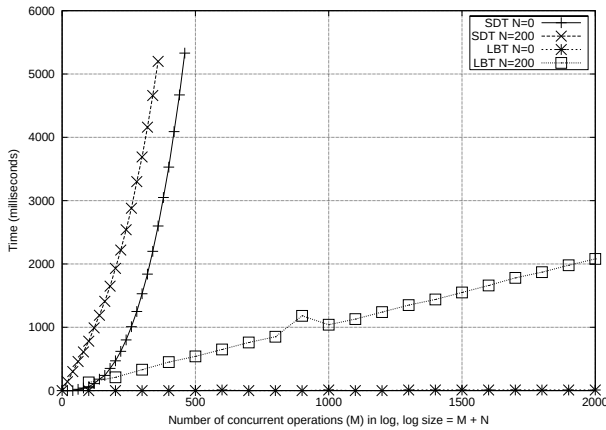


Figure 3: Comparing the worst-case performance of LBT and SDT to integrate a remote operation.

Pentium-4 2.6 GHz CPU and 2 GB RAM. Figure 3 shows the time to integrate a remote operation o into a log of n operations, $n = M + N$, in which M operations are concurrent with o and N operations happened before o . The performance advantage of LBT over SDT is obvious.

The experimental results in Figure 3 also reveal that the execution time of `Integrate()` depends on the size of and the kind of operations in HB . In [5], we show the performance of `Integrate()` affects responsiveness and awareness unless a relatively small HB is maintained in the group editor (as in [14, 16]). Hence this performance improvement is relevant.

More importantly, the approach of LBT is much simplified over that of SDT. It formally establishes the sufficient conditions of IT/ET functions. Then the control algorithm transposes the operation log such that these preconditions can be satisfied. The correctness proof is thus reduced to only that the preconditions are satisfied. Hence the approach of LBT scales well to a larger primitive operation set.

8. CONCLUSIONS

This paper proposes a novel approach called LBT for concurrency control in group editors. It formally establishes new sufficient conditions of transformation functions (IT/ET), which fix the conditions established in [14, 13]. Based on these new conditions, LBT only needs to find a control algorithm (and a particular transformation path) that ensures the correctness of IT/ET. This avoids the costs of developing OT algorithms that must verify difficult transformation properties [9] and intention preservation [11, 14] as in previous work. Therefore LBT exemplifies a more practical approach to developing OT algorithms. In addition, the control algorithm presented in this paper outperforms the state-of-the-art SDT algorithm [4, 6] in the worst case and hence provides a better guarantee for local responsiveness. In future research, we plan to extend this work to treat stringwise operations (as in [14]) and structured data (as in [1]). For space reasons, the consistency model and correctness proofs are more rigorously presented in [7].

9. REFERENCES

[1] Aguido Horatio Davis, Chengzheng Sun, and Junwei Lu. Generalizing operational transformation to the

standard general markup language. In *ACM CSCW'02*, pages 58–67, November 2002.

[2] Clarence A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *ACM SIGMOD'89 Proceedings*, pages 399–407, Portland Oregon, 1989.

[3] Abdessamad Imine, Pascal Molli, Gerald Oster, and Michael Rusinowitch. Proving correctness of transformation functions in real-time groupware. In *European CSCW'03 Proceedings*, September 2003.

[4] Du Li and Rui Li. Ensuring consistency in real-time group editors. *ACM Transactions on Computer-Human Interaction*, April 2004. Under review.

[5] Du Li and Rui Li. An operational transformation algorithm and performance evaluation. *Journal of Computer-Supported Cooperative Work*, November 2004. under review.

[6] Du Li and Rui Li. Preserving operation effects relation in group editors. In *ACM CSCW'04 Conference*, pages 457–466, November 2004.

[7] Rui Li and Du Li. A new operational transformation framework for real-time group editors. *IEEE Transactions on Parallel and Distributed Systems*, January 2005. Under review.

[8] Rui Li, Du Li, and Chengzheng Sun. A time interval based consistency control algorithm for interactive groupware applications. In *IEEE International Conference on Parallel and Distributed Systems (ICPADS)*, pages 429–436, July 2004.

[9] Matthias Ressel, Doris Nitsche-Ruhland, and Rul Gunzenhäuser. An integrating, transformation-oriented approach to concurrency control and undo in group editors. In *ACM CSCW'96 Proceedings*, pages 288–297, November 1996.

[10] Haifeng Shen and Chengzheng Sun. Flexible notification for collaborative systems. In *ACM CSCW'02*, pages 77–86, November 2002.

[11] Maher Suleiman, Michèle Cart, and Jean Ferrié. Serialization of concurrent operations in a distributed collaborative environment. In *ACM GROUP'97 Proceedings*, Phoenix, AZ, November 1997.

[12] Chengzheng Sun. Undo as concurrent inverse in group editors. *ACM Transactions on Computer-Human Interaction*, 9(4):309–361, December 2002.

[13] Chengzheng Sun and Clarence Ellis. Operational transformation in real-time group editors: issues, algorithms, and achievements. In *ACM CSCW'98*, pages 59–68, December 1998.

[14] Chengzheng Sun, Xiaohua Jia, Yanchun Zhang, Yun Yang, and David Chen. Achieving convergence, causality-preservation, and intention-preservation in real-time cooperative editing systems. *ACM Transactions on Computer-Human Interaction*, 5(1):63–108, March 1998.

[15] Nicholas Vidot, Michelle Cart, Jean Ferrie, and Maher Suleiman. Copies convergence in a distributed realtime collaborative environment. In *ACM CSCW'00*, pages 171–180, December 2000.

[16] Yun Yang, Chengzheng Sun, Yanchun Zhang, and Xiaohua Jia. Real-time cooperative editing on the Internet. *IEEE Internet Computing*, 4(3):18–25, 2000.